

Startup Orbit

System Architecture and Functionality Report

PLATFORM OVERVIEW

Executive Summary

Startup Orbit is a **centralized aggregation platform** designed to automatically source, categorize, and surface high-value opportunities for startups – including accelerators, grants, conferences, and jobs.

The system is engineered to minimize manual data entry through automated background processing, while delivering a seamless, responsive interface for end-users to discover opportunities.

Accelerators

Curated program listings

Grants

Funding opportunities

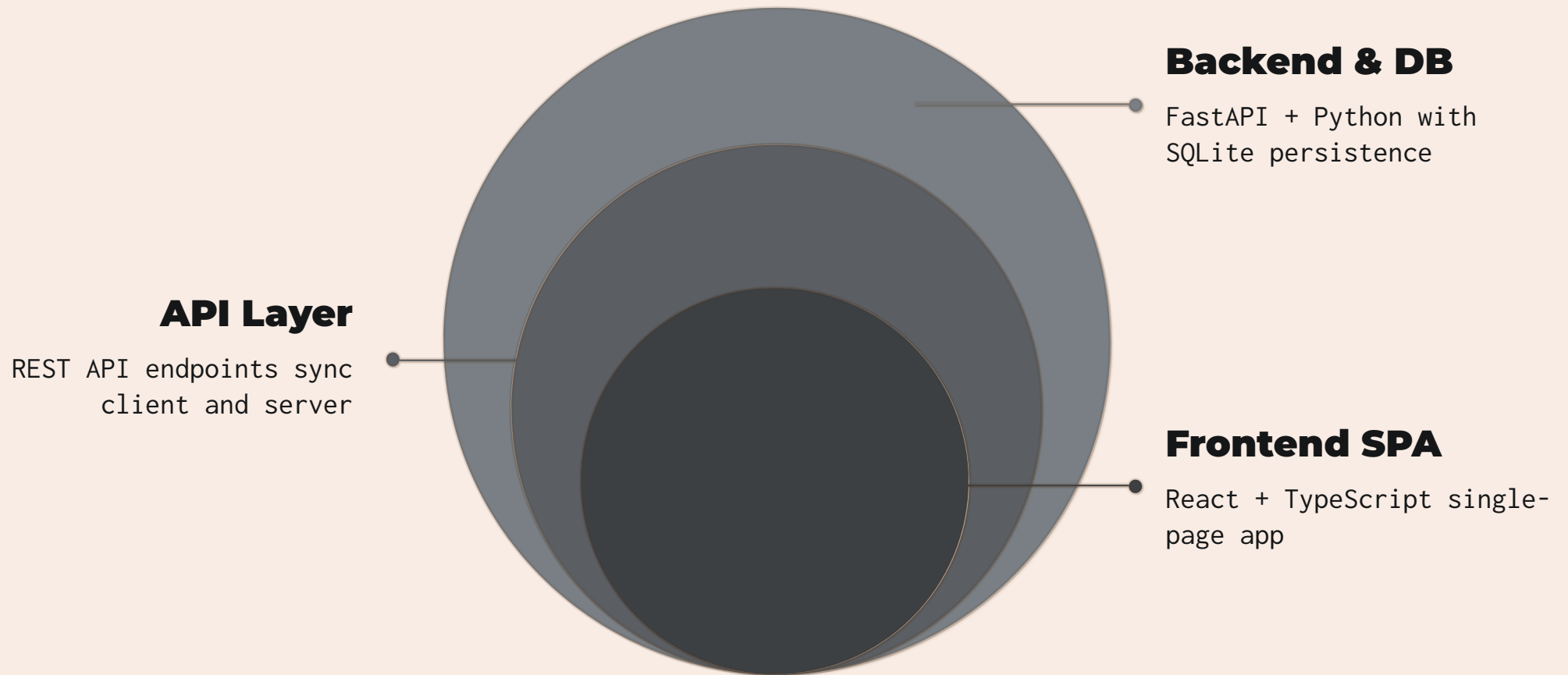
Conferences

Events and networking

Jobs

Startup-focused roles

System Architecture Overview



The platform follows a clean separation of concerns – the frontend handles user interaction, the backend manages business logic and automation, and the API layer keeps both components synchronized in real time.

Frontend Architecture

Built with **React**, **TypeScript**, and **Vite** for a highly performant single-page application experience.

Dashboard Hub

Opportunities displayed in a grid of cards showing organizer, location, funding range, and deadlines.

Dynamic Filtering

Keyword search combined with categorical filters – Type, Source, and Startup Stage – with instant state updates, no page reloads.

Pagination

Data fetched in manageable chunks to reduce bandwidth usage and maintain optimal browser memory consumption.

Administrative Interface

A **protected administrative panel** sits alongside the public dashboard, accessible only after authentication.

Admins can manually add, edit, or remove opportunity records directly – bypassing automated aggregation when precision overrides are required.

- ① Admin routes are fully isolated from public-facing endpoints, ensuring zero exposure of privileged operations to unauthenticated users.

Backend Architecture

Built on FastAPI (Python)

The backend serves as the platform's central nervous system – handling API requests, database operations, and background task scheduling with high throughput and minimal latency.

SQLite Relational Database

The **Opportunity** model stores title, type, organizer, location, deadlines, links, and categorical tags. Unique constraints on source URLs enforce automatic deduplication at the database level.

GET /opportunities

Paginated listing of
all records

GET /export

Full database export to
CSV or JSON

POST /auth

Admin session
authentication

CRUD /records

Create, Read, Update,
Delete on records

Automated Data Aggregation Engine

A background scheduler runs continuously within the backend, autonomously collecting data from external platforms without human intervention.



Job Boards

Dedicated scrapers target startup-focused job platforms, extracting structured listing data on a recurring schedule.



RSS Feeds

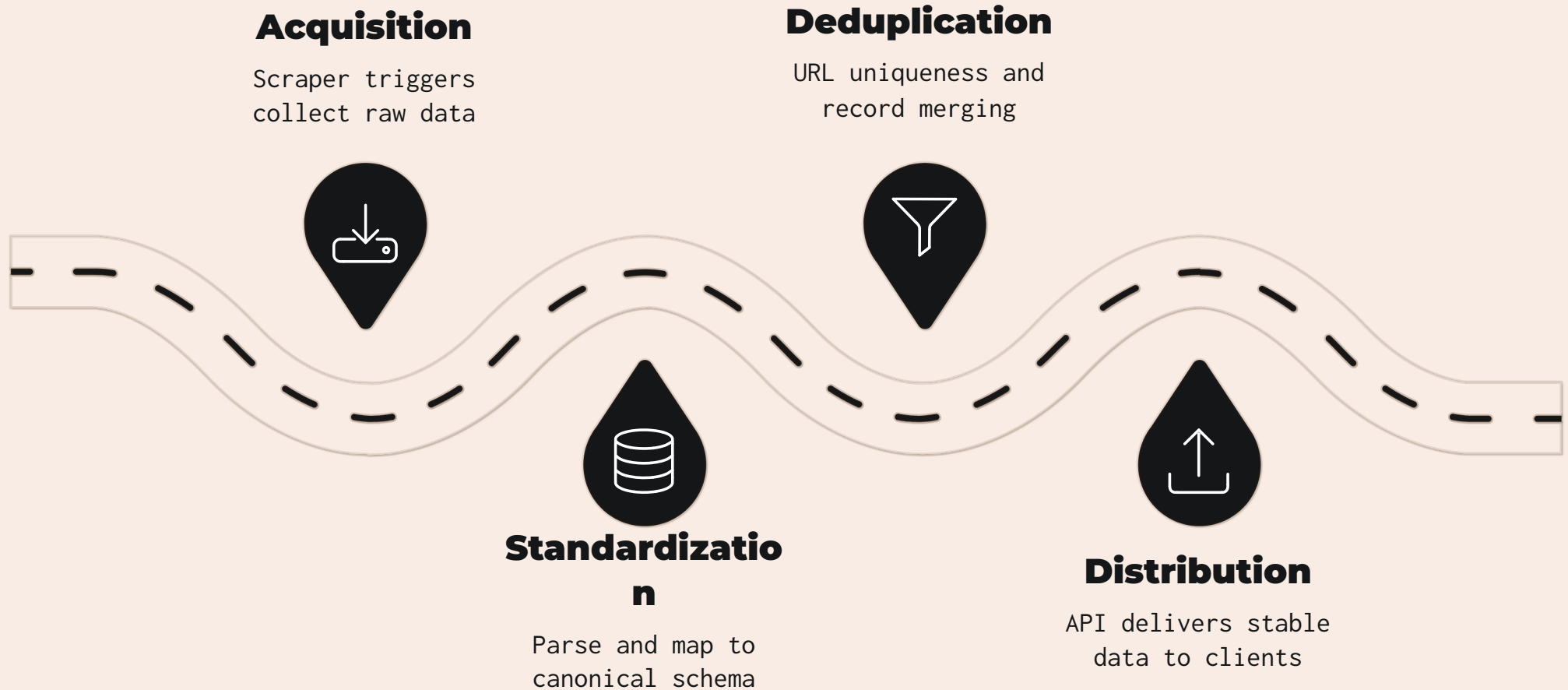
Startup program and news feeds are parsed and normalized into the internal data schema automatically.



Fallback Dataset

A hardcoded set of verified, high-value opportunities guarantees the database is never empty – even if live scraping temporarily fails.

Data Lifecycle Workflow



Each stage is decoupled – scraping failures are isolated to the acquisition layer and never propagate to the distribution or presentation layers, ensuring consistent uptime for end-users.

Security and Authentication

Defense-in-Depth Approach

Security is applied at every layer of the admin stack – from credential verification to cookie policy.

- **Credential Verification:** Login attempts are validated against hashed credentials stored in the database or environment variables.
- **HTTP-Only Cookies:** Issued on successful auth, mitigating XSS attack vectors for subsequent admin requests.
- **Endpoint Isolation:** All administrative routes are fully protected and inaccessible without a valid active session.

Conclusion

A Self-Sustaining Architecture

Startup Orbit decouples automated data acquisition from the user-facing presentation layer – enabling high performance, resilience, and reliability without manual maintenance overhead.

→ Resilient Scraping

Error-handling and fallback datasets ensure zero downtime for end-users.

→ Structured Data Models

Relational schema with deduplication enforces integrity at the database level.

→ Responsive Frontend

React/TypeScript SPA delivers a fast, filterable interface for both users and admins.